

1. Funções Customizadas (Modularização)

Até agora, seus alunos programaram escrevendo linhas de código direto dentro do `setup()` e do `loop()`. Para projetos maiores, isso deixa o código confuso e difícil de corrigir. A solução é criar **Funções**.

Uma função é um bloco de código isolado que realiza uma tarefa específica e pode ser chamado a qualquer momento.

1.1 Estrutura de uma Função

Uma função possui três partes principais:

1. **Tipo de Retorno:** O tipo de dado que ela devolve para quem a chamou (ex: `int`, `float`). Se não devolver nada, usamos `void`.
2. **Nome e Parâmetros:** O nome da função e os dados que ela precisa receber para trabalhar (variáveis locais entre parênteses).
3. **Corpo:** O código que será executado.

C++

// Exemplo de função que calcula a média de dois valores analógicos

```
float calcularMedia(int valor1, int valor2) {  
    float resultado = (valor1 + valor2) / 2.0;  
    return resultado; // Devolve o valor calculado  
}
```

```
void loop() {
```

```
    int sensorA = analogRead(A0);
```

```
    int sensorB = analogRead(A1);
```

```
    // Chamando a função e salvando o retorno em uma variável
```

```
    float mediaSensores = calcularMedia(sensorA, sensorB);
```

```
    delay(1000);
```

```
}
```

2. Funções Nativas: Matemática, Trigonometria e Texto

A linguagem C/C++ do ambiente Arduino possui diversas funções prontas para economizar tempo de desenvolvimento:

- **Matemáticas e Trigonométricas:**
 - `abs(x)`: Retorna o valor absoluto (módulo) de um número (transforma negativo em positivo).
 - `pow(base, expoente)`: Calcula a potência de um número.
 - `sqrt(x)`: Calcula a raiz quadrada.
 - `sin(rad)`, `cos(rad)`, `tan(rad)`: Funções trigonométricas (valores em radianos).
 - `map(valor, deBaixo, deAlto, paraBaixo, paraAlto)`: **A mais importante para sensores!** Redimensiona um valor de uma faixa para outra (ex: transformar a leitura do LDR de 0-1023 para brilho de LED de 0-255).
- **Manipulação de Texto (Strings):**
 - `String objetoTexto = "Etec";`
 - `objetoTexto.length()`: Retorna o tamanho do texto (quantidade de caracteres).
 - `objetoTexto.toUpperCase()`: Transforma todo o texto em letras maiúsculas.
 - `objetoTexto.concat(" CPS")`: Junta (concatena) textos.

3. Geração de Números Aleatórios

Muito útil para criar jogos (como o Genius/Simon Says), simular ruídos ou alternar comportamentos de robôs de forma imprevisível.

- `random(max)`: Gera um número aleatório entre 0 e (max - 1).
- `random(min, max)`: Gera um número aleatório entre o valor mínimo e (max - 1).

O Pulo do Gato (randomSeed): Computadores não geram números verdadeiramente aleatórios, eles usam uma tabela matemática fixa. Se você ligar o Arduino, a sequência de números gerada pelo `random()` será **sempre a mesma** toda vez que reiniciar. Para resolver isso, usamos `randomSeed(analogRead(A5))` no `setup()`. Como o pino analógico A5 estará desconectado (flutuando), ele captará um ruído eletromagnético aleatório do ambiente, servindo como uma "semente" verdadeiramente única para embaralhar a tabela.

4. Emissão de Sons com Buzzer (Função tone)

O **Buzzer** é um pequeno componente piezoelétrico que emite som ao receber uma frequência elétrica alternada.

Para controlar a frequência (nota musical) e o tempo do som, usamos a função nativa `tone()`.

4.1 Sintaxe da Função

- `tone(pino, frequencia);` // Emite a frequência por tempo indeterminado.
- `tone(pino, frequencia, duracao);` // Emite a frequência com tempo definido em milissegundos.
- `noTone(pino);` // Força o pino a parar de emitir som.

Exemplo de frequências das notas musicais comuns:

- **Dó (C4):** 262 Hz
- **Lá (A4):** 440 Hz (*Padrão de afinação*)
- **Dó (C5):** 523 Hz

C++

```
const int pinoBuzzer = 9;
```

```
void setup() {  
  randomSeed(analogRead(A5));  
}
```

```
void loop() {  
  // Gera uma frequência aleatória entre 200Hz e 1000Hz  
  int frequenciaAleatoria = random(200, 1001);  
  
  tone(pinoBuzzer, frequenciaAleatoria, 150); // Toca por 150ms  
  delay(300); // Espera o tempo do som + pausa  
}
```

Exercícios de Fixação

Exercício 1: Escalonamento de Sensores. Um aluno está lendo um sensor de temperatura cuja saída analógica varia de **200 a 800** no `analogRead()`. Ele precisa converter essa leitura para uma escala de porcentagem de calor que vai de **0% a 100%**. Escreva a linha de código utilizando a função `map()` para realizar essa conversão de escala de forma correta.

Exercício 2: Criação de Função Customizada. Crie uma função em C++ chamada `piscarAlerta`. Ela deve ser do tipo `void`, receber como parâmetro um número inteiro correspondente ao `pinoLED` e não retornar nenhum valor. O corpo da função deve apenas acender o LED recebido, esperar 100ms, apagá-lo e esperar mais 100ms.

Exercício 3: Lógica de Áudio e Aleatoriedade. Explique detalhadamente qual é o efeito físico produzido no circuito (o que o usuário vai ouvir e ver) ao executar o trecho de código abaixo dentro do `loop()` do Arduino:

C++

```
int pinoLED = 13;
```

```
int pinoBuzzer = 9;
```

```
void loop() {
```

```
    int tempo = random(100, 1000);
```

```
    digitalWrite(pinoLED, HIGH);
```

```
    tone(pinoBuzzer, 440);
```

```
    delay(tempo);
```

```
    digitalWrite(pinoLED, LOW);
```

```
    noTone(pinoBuzzer);
```

```
    delay(tempo);
```

```
}
```

Respostas (Para o professor):

1. `int porcentagem = map(valorLido, 200, 800, 0, 100);`
2. Código da função esperada:

C++

```
void piscarAlerta(int pinoLED) {  
    digitalWrite(pinoLED, HIGH);  
    delay(100);  
    digitalWrite(pinoLED, LOW);  
    delay(100);  
}
```

3. *O usuário ouvirá a nota musical Lá (440Hz) saindo do buzzer ao mesmo tempo em que o LED do pino 13 acende. O tempo que o LED e o som ficam ligados (e depois o tempo que ficam desligados) mudará de forma aleatória a cada ciclo do loop, piscando e bipando em um ritmo completamente irregular (entre 100ms e 1 segundo).*